第 3 章 Bonus：高效 Multi-Head Attention 学习教程

基于

`ch03/02_bonus_efficient-multihead-attention/mha-implementations.ipynb`

2026 年 4 月 17 日

目录

1 这份教程解决什么问题 **3**

2 先看整条主线 **3**

3 统一问题设定 **4**

3.1 所有实现都在做同一件事 5

4 实现一：第 3 章的 head wrapper 版 **5**

4.1 它为什么最直观 5

4.2 它慢在哪里 5

5 实现二：第 3 章的单模块 MHA **6**

5.1 这才是主章节真正的工程基线 6

5.2 它比 wrapper 版好在哪 6

6 实现三：合并 QKV 投影 **6**

6.1 为什么这一步很自然 6

6.2 它解决了什么 7

6.3 理解时要注意的点 7

7 实现四：`einsum` 版 MHA **7**

7.1 它在教学上有什么价值 7

7.2 但它不一定是性能赢家 7

7.3 为什么仓库专门给它写测试 8

8 实现五：PyTorch 的 SDPA 与 FlashAttention 路线 **8**
8.1 为什么这是现代实现的分水岭 8
8.2 为什么它往往更快 8
8.3 这里最值得记住的一点 9

9 实现六：显式 mask 的 SDPA **9**
9.1 为什么它还要单独比较 9
9.2 它揭示的工程事实 9

10 实现七与实现八：`nn.MultiheadAttention` **9**
10.1 为什么默认版不一定快 9
10.2 这个细节为什么重要 10

11 实现九：FlexAttention **10**
11.1 它想解决的是什么问题 10
11.2 它的现实限制 10

12 把 9 种实现放到一张图里 **11**

13 怎么读 benchmark 结果 **11**
13.1 结论一：教学友好的写法通常不是最快 11
13.2 结论二：一旦进入 fused attention 路线，性能会明显改善 12
13.3 结论三：benchmark 不是永久真理 12

14 为什么 notebook 还比较 forward/backward 和 compile **12**
14.1 只看 forward 很容易误导 12
14.2 `torch.compile` 会改变排名 12

15 如果你自己要实现，该怎么选 **12**
15.1 场景一：你在学 attention 12
15.2 场景二：你要写一个可维护的教学型项目 13
15.3 场景三：你要做实际训练或推理 13
15.4 场景四：你要支持更复杂注意力模式 13

16 这一份 bonus 最该学会什么 **13**

17 下一步可以继续看什么 **13**

1 这份教程解决什么问题

主章节 `ch03/01_main-chapter-code/ch03.ipynb` 已经把 attention 从零推到了可训练的 multi-head attention。这个 bonus notebook 要回答的不是 “attention 是什么”，而是更工程化的问题：

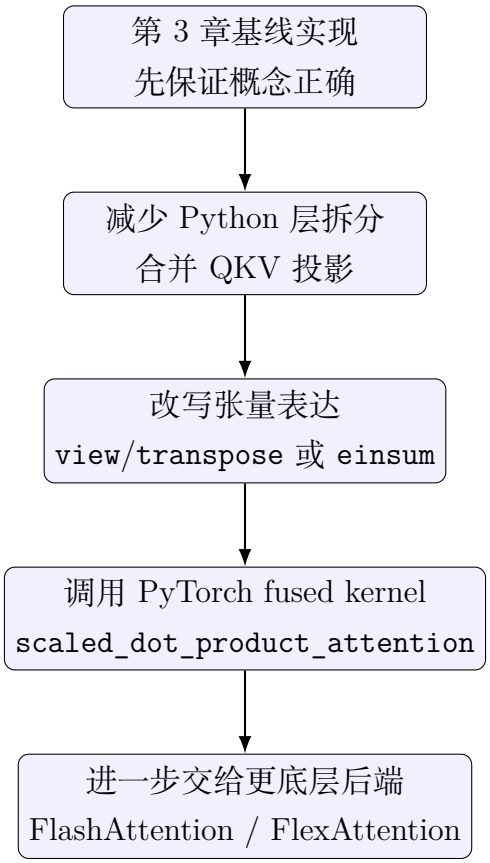
> 当你已经知道 causal multi-head attention 的数学定义后，应该如何在 PyTorch 里把它写得更快、更省内存、更接近现代 LLM 的实际实现？

所以这份中文教程的目标是把 notebook 里的多种实现方式重新组织成一条清晰主线：

1. 第 3 章里最朴素的 MHA 实现到底慢在什么地方；
2. 为什么合并 QKV 投影通常比三套线性层更自然；
3. `einsum` 版和矩阵变形版本质上是不是一回事；
4. PyTorch 的 `scaled_dot_product_attention` 为什么往往更快；
5. `nn.MultiheadAttention` 的默认行为和 “高性能模式” 差在哪；
6. FlexAttention 解决了什么问题，又有哪些现实限制；
7. 面对训练、推理、教学、调试四种不同目标时，该选哪一种实现。

2 先看整条主线

这份 bonus 的核心不是记住 8 个类名，而是看清楚一条优化链：



阅读时如果觉得类很多、名字很像，就记住这条线：**同一个 causal MHA 数学对象，在实现层面经历的是“少做 Python 拆分”→“更好地配合底层 kernel”→“把注意力计算交给框架优化器”**。

3 统一问题设定

notebook 一开始固定了一组对比配置：

- batch size = 8；
- context length = 1024；
- embedding dimension = 768；
- 头数 = 12；
- dropout 在 benchmark 中设为 0；
- 注意力类型统一是 decoder-style 的 causal self-attention。

因此，这个 bonus 不是在比较不同模型结构，而是在比较 **同一个 MHA 计算图被不同 API 和不同张量写法实现时的工程代价**。

3.1 所有实现都在做同一件事

无论你用哪一种写法，核心计算都没变：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_{\mathrm{head}}}} + M_{\mathrm{causal}}\right) V$$

差异主要来自三件事：

1. Q、K、V 是分三次投影还是一次性投影；
2. 分头、转置、mask 和 softmax 是不是由你手工拼出来；
3. 最耗时的 attention kernel 是由 Python 拼接，还是直接交给 PyTorch fused operator。

4 实现一：第 3 章的 head wrapper 版

4.1 它为什么最直观

第一种实现 `Ch03_MHA_Wrapper` 直接把每个头写成一个 `CausalAttention`，再用 `ModuleList` 逐头调用后拼接：

```
context_vec = torch.cat([head(x) for head in self.heads], dim=-1)
return self.out_proj(context_vec)
```

这版最适合教学，因为它保留了“一个 head 就是一套完整 causal attention”的直觉。你几乎可以肉眼看到：

1. 每个 head 先各自做 Q, K, V 投影；
2. 每个 head 各自做 mask、softmax、加权求和；
3. 最后把多个 head 拼起来，再做输出投影。

4.2 它慢在哪里

不过这版也正是最不适合真实训练的写法，因为它在 Python 层做了太多重复工作：

1. 每个 head 都有独立模块和独立前向调用；
2. 每个 head 各自触发一轮线性层和一轮注意力计算；
3. `torch.cat` 之前，所有头的结果都是分散管理的。

所以它的价值主要是**教学清晰**，不是**运行高效**。

5 实现二：第 3 章的单模块 MHA

5.1 这才是主章节真正的工程基线

`Ch03_MHA` 把所有头放进一个模块里，先整体投影，再通过 `view` 和 `transpose` 把最后一维拆成 `num_heads` × `head_dim`：

```
keys = keys.view(b, num_tokens, self.num_heads, self.head_dim)
keys = keys.transpose(1, 2)
```

这里最关键的工程转变是：

> multi-head attention 不是“很多小模块并联”，而是“一个大张量在多头维度上重排后统一计算”。

5.2 它比 wrapper 版好在哪

这版的优势主要有三点：

1. 参数管理更集中，只有一套 W_Q, W_K, W_V 和一套输出投影；
2. 每个 head 的计算都被并进了批量张量运算；
3. 更接近后续 fused kernel 能吃下去的数据布局。

如果你正在学 attention，`Ch03_MHA` 是最值得吃透的一版。因为从它往后，几乎所有优化都只是“不改数学定义，只改落地方式”。

6 实现三：合并 QKV 投影

6.1 为什么这一步很自然

notebook 的第三种实现把三套线性层合成一套：

```
self.qkv = nn.Linear(d_in, 3 * d_out, bias=qkv_bias)
```

然后再 reshape 成 $(b, n, 3, h, d_{\text{head}})$，最后拆出 queries、keys、values。
这一步的直觉非常简单：

> 既然输入 X 一样，只是输出被分成 Q、K、V 三块，那就没有必要让 Python 连续调用三次独立线性层。

6.2 它解决了什么

合并 QKV 的主要收益不是改变复杂度阶数，而是减少实现常数项：

1. 更少的模块调用开销；
2. 更连续的内存访问；
3. 更符合现代 Transformer 库的常见参数布局；
4. 后续更容易接到 fused attention kernel。

6.3 理解时要注意的点

这里不要把 “合并 QKV” 理解成 “数学上少做了一步”。真正发生的是：

$$XW_Q,\ XW_K,\ XW_V \quad \Longrightarrow \quad X[W_Q\ W_K\ W_V]$$

也就是说，它只是把三次线性变换拼成了一次更大的线性变换，本质仍然是同一套投影。

7 实现四：einsum 版 MHA

7.1 它在教学上有什么价值

`MHAEinsum` 用 `torch.einsum` 把线性变换和注意力打分写成显式的指标求和：

```
Q = torch.einsum("bnd,do->bno", x, self.W_query)
scores = torch.einsum("bhnd,bhmd->bhnm", Q, K)
```

这版的好处是表达非常直接。你可以一眼看到：

1. 哪个维度是 batch；
2. 哪个维度是 token；
3. 哪个维度是 head；
4. 哪个维度在做缩并。

所以它很适合做**张量维度教学**和**实现正确性对照**。

7.2 但它不一定是性能赢家

`einsum` 的问题不在于它错，而在于它未必总能落到最优 kernel。在很多场景下，PyTorch 能把它优化得不错，但它通常没有 `scaled_dot_product_attention` 那样明确地走到专门的注意力后端。

7.3 为什么仓库专门给它写测试

配套测试文件 `ch03/02_bonus_efficient-multihead-attention/tests/test_mha_implementati` 专门检查了 `MHAEinsum` 和 `Ch03_MHA` 在拷贝权重后的输出是否一致：

```
assert torch.allclose(out_linear, out_einsum, atol=1e-5)
```

这说明仓库作者也把它当作一种“表达形式不同、数学对象相同”的实现，而不是另一种 attention 机制。

8 实现五：PyTorch 的 SDPA 与 FlashAttention 路线

8.1 为什么这是现代实现的分水岭

从这一步开始，重点已经不再是“你能不能自己写出 attention”，而是“你能不能把数据排布成 PyTorch fused kernel 喜欢的样子”。

`MHAPyTorchScaledDotProduct` 调用的是：

```
nn.functional.scaled_dot_product_attention(
    queries, keys, values,
    attn_mask=None,
    dropout_p=use_dropout,
    is_causal=True
)
```

这行代码的重要性非常高，因为它可能触发 PyTorch 背后的高性能实现，包括 memory-efficient attention 或 FlashAttention 风格后端。

8.2 为什么它往往更快

原因不是“库函数更高级”，而是它给了底层足够多的信息：

1. 这是标准的 scaled dot-product attention；
2. 这是 causal 场景；
3. mask 可以不显式物化成大矩阵；
4. softmax、mask、乘法、归约可以由 fused kernel 一起做。

这通常会带来两类收益：

1. 更少的中间张量读写；
2. 更低的显存带宽压力。

8.3 这里最值得记住的一点

如果你只是想在现代 PyTorch 里写一个实用版 causal MHA，**优先考虑 SDPA 路线，而不是自己手工写 `attn_scores`、`masked_fill`、`softmax`。**

9 实现六：显式 mask 的 SDPA

9.1 为什么它还要单独比较

`MHAPyTorchSDPAWithoutFlash` 仍然调用 SDPA，但传入了显式的 causal mask，同时把 `is_causal=False`：

```
context_vec = nn.functional.scaled_dot_product_attention(
    queries, keys, values,
    attn_mask=attn_mask,
    dropout_p=use_dropout,
    is_causal=False
)
```

这样做的意义在于把两个问题拆开：

1. “我是否使用 SDPA 这个 API”；
2. “我是否让后端知道这是一个标准 causal 模式”。

9.2 它揭示的工程事实

同一个 API，在你是否显式传 mask、是否直接声明 `is_causal=True` 的情况下，底层可用优化路径可能不同。

这正是 notebook 想强调的一点：

> 高性能实现不只是 “用了哪个函数”，还取决于你是否以最容易被框架识别和融合的方式使用它。

10 实现七与实现八：`nn.MultiheadAttention`

10.1 为什么默认版不一定快

notebook 又比较了两种 PyTorch 官方类封装：

1. 默认的 `nn.MultiheadAttention`；
2. 把 `need_weights=False` 的版本。

这两个版本最值得学习的不是 API 本身，而是一个工程细节：

> 如果你要求返回 attention weights，框架可能不能走最优的 SDPA 路线。

所以 notebook 专门强调：

```
need_weights=False
```

10.2 这个细节为什么重要

很多训练和推理场景根本不需要把 attention map 返回给上层。如果你只是想拿最终输出，而不是做可视化或解释性分析，那么保留 `need_weights=True` 往往只是平白限制了后端优化空间。

因此，**官方类并不等于自动最优**；你仍然要理解哪些参数会让它退回更慢的路径。

11 实现九：FlexAttention

11.1 它想解决的是什么问题

FlashAttention 很快，但它支持的 mask 和模式往往更固定。FlexAttention 的目标是：

> 尽量保留 PyTorch 级别的灵活性，同时提供接近 FlashAttention 的性能。

notebook 里用 `create_block_mask` 和一个 `causal` 谓词来构造 block mask，再交给 `flex_attention`：

```
attn_mask = create_block_mask(
    causal, B=None, H=None,
    Q_LEN=num_tokens, KV_LEN=num_tokens,
    device=x.device
)
context_vec = flex_attention(queries, keys, values, block_mask=
   attn_mask)
```

11.2 它的现实限制

notebook 也明确写了两个重要前提：

1. 需要 PyTorch 2.5 或更新版本；
2. 当前主要支持 CUDA，且代码里提到 dropout 仍有限制。

另外，它还保留了一段被注释掉的旧写法，解释为什么现在要在 forward 中重新创建 mask：某些新版本下，`BlockMask` 已不支持旧的切片方式。

这提醒你一个很实际的问题：**越靠近新内核和新特性，版本兼容性成本往往越高**。

12 把 9 种实现放到一张图里

实现	核心思路	主要优点	主要代价
Chapter 3 wrapper	每个 head 一个模块	教学最直观	Python 开销大
Chapter 3 MHA	单模块统一分头	易懂且已较实用	仍手工写 attention 流程
Combined QKV	QKV 合并投影	更少线性层调用	代码可读性略降
`einsum`	显式指标缩并	维度语义清楚	未必落到最优 kernel
SDPA + causal flag	直接调 fused API	现代 PyTorch 常用优选	需要理解后端约束
SDPA + explicit mask	仍用 SDPA，但显式传 mask	方便对照 mask 影响	可能错过最快路径
`nn.MultiheadAttention` 默认	官方封装最省事	API 稳定	默认不一定最快
`nn.MultiheadAttention` + `need_weights=False`	官方类更接近 SDPA 最优路径	常常更高效	不再返回注意力权重
FlexAttention	更灵活的高性能 mask 机制	适合更复杂模式扩展	版本和平台要求更高

13 怎么读 benchmark 结果

README 给出了三张总结图，分别比较：

1. 仅 forward；
2. forward + backward；
3. forward + backward + `torch.compile`。

理解这些结果时，不要只盯着“谁是第一名”，而是要抓三层结论。

13.1 结论一：教学友好的写法通常不是最快

wrapper 版和手工展开版更容易学，但在大序列长度下，Python 层拆分、显式 mask、中间张量读写都会形成明显开销。

13.2 结论二：一旦进入 fused attention 路线，性能会明显改善

只要实现方式能够让 PyTorch 识别出标准 SDPA 结构，就更有机会使用优化后的 kernel。这也是为什么现代框架越来越不鼓励手写 `masked_fill` + `softmax` + `@ values` 的原因。

13.3 结论三：benchmark 不是永久真理

这一点尤其重要。notebook 结果依赖于：

1. 具体硬件，是 M3 还是 A100；
2. 具体 PyTorch 版本；
3. 是否启用 `torch.compile`；
4. batch、序列长度、头数和 dropout 配置；
5. 是否处于训练模式。

所以正确的态度不是背结论，而是学会**如何设计一个公平 benchmark**。

14 为什么 notebook 还比较 forward/backward 和 compile

14.1 只看 forward 很容易误导

推理场景当然关心 forward，但训练真正消耗时间的通常是 forward + backward。某些实现前向快，不代表反向也快。

14.2 `torch.compile` 会改变排名

`torch.compile` 的价值在于让 PyTorch 有机会进一步融合图和优化执行。但不是所有实现都能从中得到同样收益。

因此 notebook 把 **裸跑**和 **compile 后**分开比较，是很合理的设计。这也提醒你：**工程上的最快实现，常常是“实现写法”和“编译/运行时配置”共同作用的结果**。

15 如果你自己要实现，该怎么选

15.1 场景一：你在学 attention

先看 `Ch03_MHA_Wrapper` 和 `Ch03_MHA`。前者帮你保留“每个 head 各算各的”的直觉，后者帮你理解真正的张量分头视角。

15.2 场景二：你要写一个可维护的教学型项目

优先选 `Ch03_MHA` 或 combined QKV 版。它们在可读性和工程合理性之间比较平衡。

15.3 场景三：你要做实际训练或推理

优先考虑 SDPA 路线，必要时再评估 `nn.MultiheadAttention` 是否能满足你的封装需求。只要不需要返回 attention weights，就尽量避免让框架多做无用工作。

15.4 场景四：你要支持更复杂注意力模式

这时可以关注 FlexAttention，因为它的优势不是“基础 causal attention 更简单”，而是“复杂 mask / block 规则下还有机会保持高性能”。

16 这一份 bonus 最该学会什么

如果只留三句话，我建议记住下面三句：

1. multi-head attention 的数学定义基本不变，真正变化的是张量布局和 kernel 调用方式；
2. 现代 PyTorch 里，标准 causal attention 优先走 SDPA 路线，而不是手工拼原语；
3. benchmark 结论只有放回硬件、版本、训练模式和编译配置里看，才有意义。

17 下一步可以继续看什么

如果你已经读完这份 bonus，下一步建议是：

1. 回到第 3 章主教程，对照理解“概念正确”和“实现高效”的差别；
2. 自己把 `Ch03_MHA` 改写成 combined QKV 或 SDPA 版；
3. 用你自己的 GPU 和 PyTorch 版本重跑 benchmark，而不是直接复用 notebook 结论；
4. 如果之后要接 KV cache、paged attention 或 block-sparse attention，再继续沿着“更接近底层 kernel”的方向深入。